

Module: Les technologies du Middleware



Chapitre 1: Introduction aux Middlewares



- Systèmes et applications répartis.
- Middleware (intergiciel) :Définition.
- Objectifs de l'intergiciel
- Catégories des intergiciels



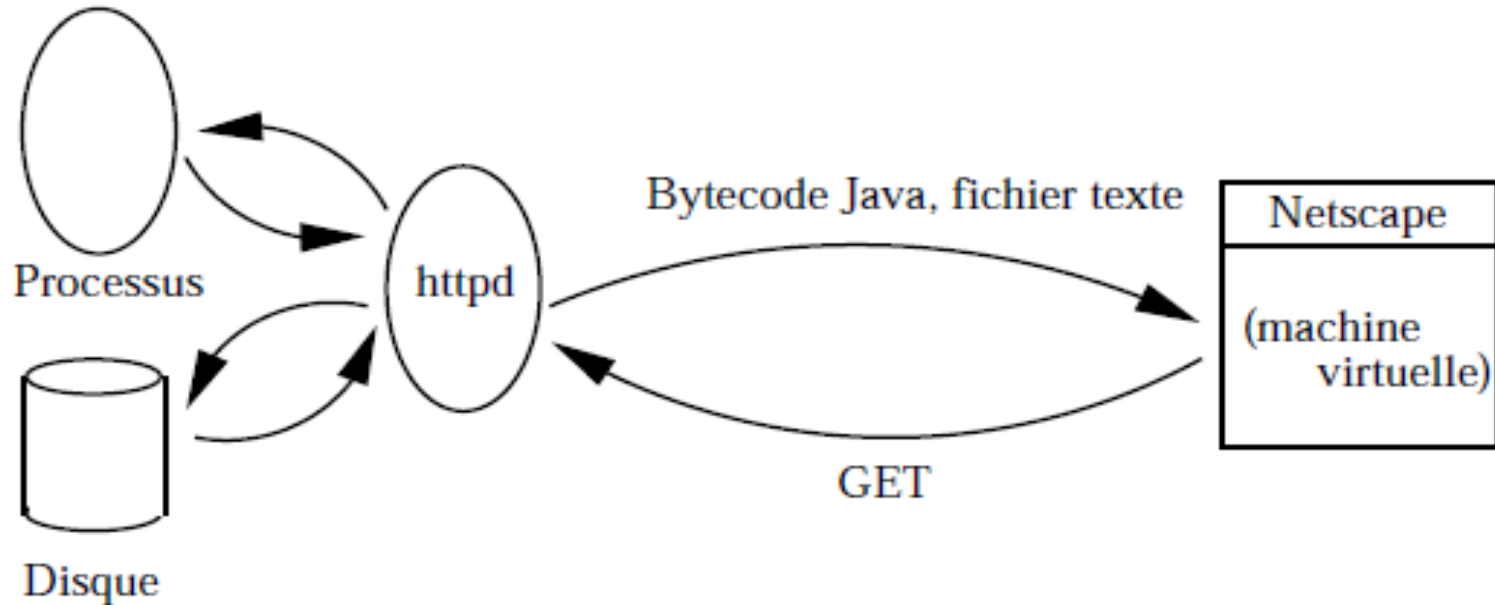
- Un système réparti est un ensemble de machines autonomes connectées par un réseau, et équipées d'un logiciel dédié à la coordination des activités du système ainsi qu'au partage de ses ressources.[1]
- Un système réparti est un système qui s'exécute sur un ensemble de machines sans mémoire partagée, mais que pourtant l'utilisateur voit comme une seule et unique machine.[2]



- Partage des ressources (données, applications, périphériques chers). Optimisation de leur utilisation
- Tolérance aux pannes (fiabilité, disponibilité).
- Contraintes physiques (ex : avionique, usines automatisées).
- Facilite la communication entre utilisateurs.
- Concurrence, parallélisme $\Rightarrow$ efficacité.
- Flexibilité, facilité d'extension du système (matériels, logiciels).



- **Transparence à la localisation:** L'utilisateur ignore la situation géographique des ressources.
- **Transparence d'accès:** L'utilisateur accède à une ressource locale ou distante d'une façon identique.
- **Transparence à l'hétérogénéité:** L'utilisateur n'a pas à se soucier des différences matérielles ou logicielles des ressources qu'il utilise.
- **Notion d'interopérabilité.**
- **Transparence aux pannes (réseaux, machines, logiciels).** Les pannes et réincarnations sont cachées à l'utilisateur. Transparence à la réplication.
- **Transparence à l'extension des ressources.** Extension ou réduction du système sans occasionner de gêne pour l'utilisateur (Sauf performance).



- Transparence à la localisation : liens hypertexte.
- Transparence d'accès : URL (site $\neq$ adresse IP).
- Désignation : URL, DNS Internet.
- Interopérabilité : Java bytecode et machine virtuelle, pages HTML.



■ **Application répartie** = traitements coopérants sur des données réparties.

■ **Coopération** = communications + synchronisation :

- Un modèle d'exécution,
- Une interface de programmation,
- Des outils de développement.

■ **Différents types de distribution** :

- Distribution des données : données distribuées, traitement centralisé.
- Distribution du contrôle : données centralisées, contrôle distribué.
- Distribution des utilisateurs : données et contrôle centralisés,
- utilisateurs distribués.

une combinaison de tous ça...



■ Coordination d'activités :

- Systèmes à flots de données (« workflow »).
- Systèmes à agents.

■ Communication et partage d'informations :

- Bibliothèques virtuelles.

■ Collecticiels :

- Édition coopérative.
- Téléconférence.

■ Applications temps réel :

- Contrôle de procédés industriels.
- Avionique, etc.
- Localisation de mobiles.
- Toutes les applications qui nécessitent des utilisateurs ou des données réparties.



- ▶ Tolérance aux pannes.
- ▶ Passage à l'échelle.
- ▶ Nommage et accès aux applications.
- ▶ Intégration de l'existant.
- ▶ Déploiement des applications.
- ▶ Sécurité et authentification.
- ▶ Disponibilité de l'application.



En anglais : reliability, fault tolerance.

- Que se passe-t-il quand :
 - Un serveur participant à l'application tombe en panne.
 - Un serveur envoie des informations erronées.
 - Un serveur n'est plus atteignable (mais pas en panne) puis le redevient.



En anglais : scalability.

- Ce qui marche pour un utilisateur, marchera-t-il pour 10 000 ?
- Ce qui marche pour un objet, marchera-t-il pour 1 000 000 ?
- Ce qui marche pour un site, marchera-t-il pour 1 000 ?
- Exemple : les applications de gestion de commerce électronique.



En anglais : Naming.

- Comment retrouver les objets distants qui sont disponibles ?
- Un objet = un identifiant + un état + un comportement.
- Dans les applications non réparties, le nommage est géré par le langage (référence) ou par l'OS (adressage).
- Dans les applications réparties, le nommage doit être explicite et dynamique.
- Exemple de nommage dans des applications répartie : DNS, URL,...



- Comment installer tous les composants logiciels sur différents clients et serveurs ?
- Lorsque je change un nom de serveur ou que j'en ajoute un, est-ce que je dois recompiler ? est-ce que je dois redéployer ? Ou est-ce que je peux configurer automatiquement le redéploiement ?



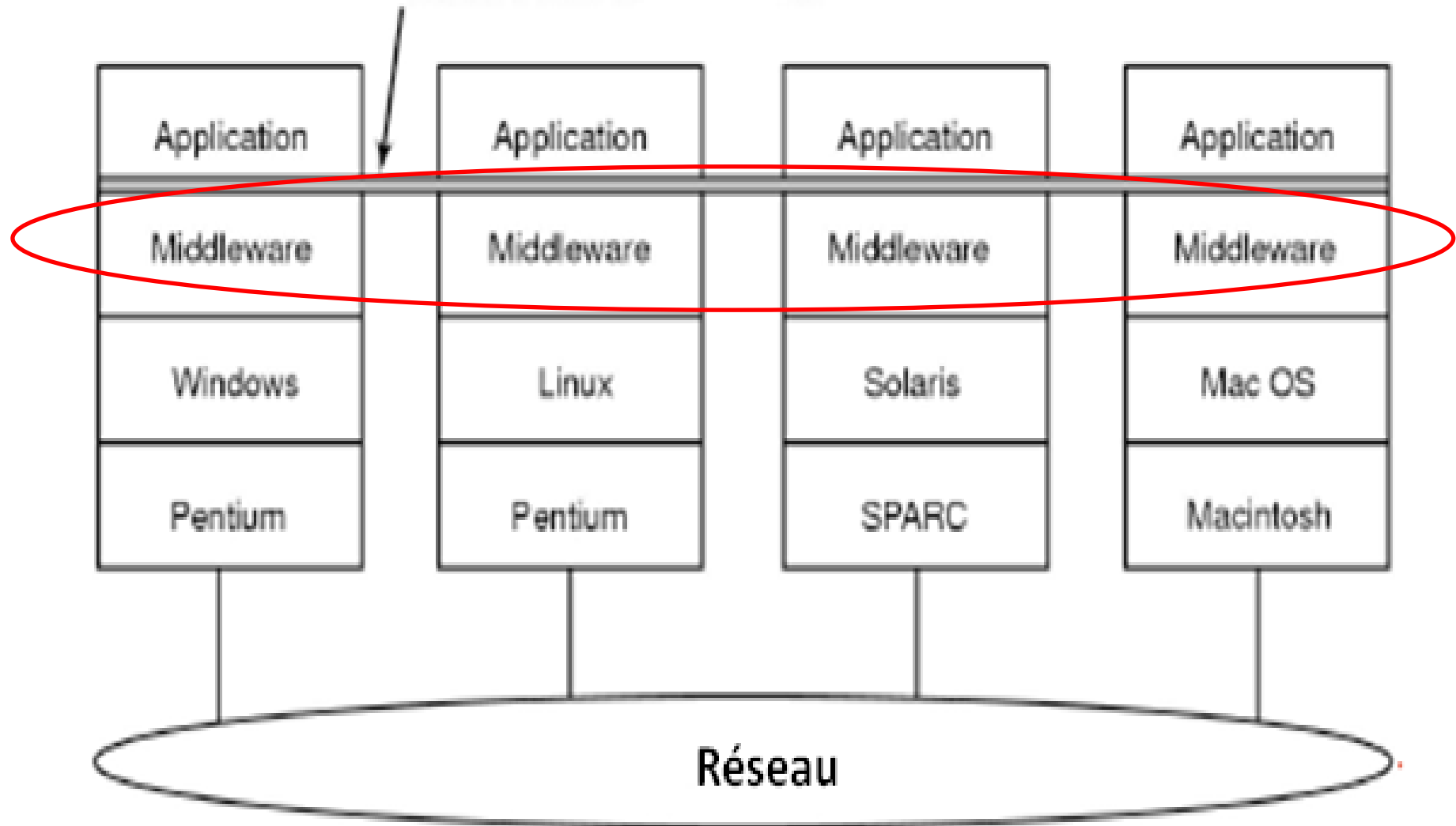
- Confidentialité.
- Intégrité :
 - Droits d'accès, Pare-Feu.
- Authentification :
 - Identification des applications partenaires.
 - Non-répudiation.
 - Messages authentifiés.
- Combien de personnes utilisent l'application, qui sont-ils ?
 - Nécessité de se protéger contre les intrusions.
 - Nécessité de stocker les accès des clients dans des fichiers Journaux.



- Exemple : un serveur qui fait de la tolérance aux pannes ne peut plus assurer d'autres tâches.
- Permettre des accès simultanés sur un même objet :
 - Sérialiser les objets.
 - Paralléliser les accès.
 - Appliquer différentes politiques.
 - Multi-threading.



Une base commune à toutes les applications

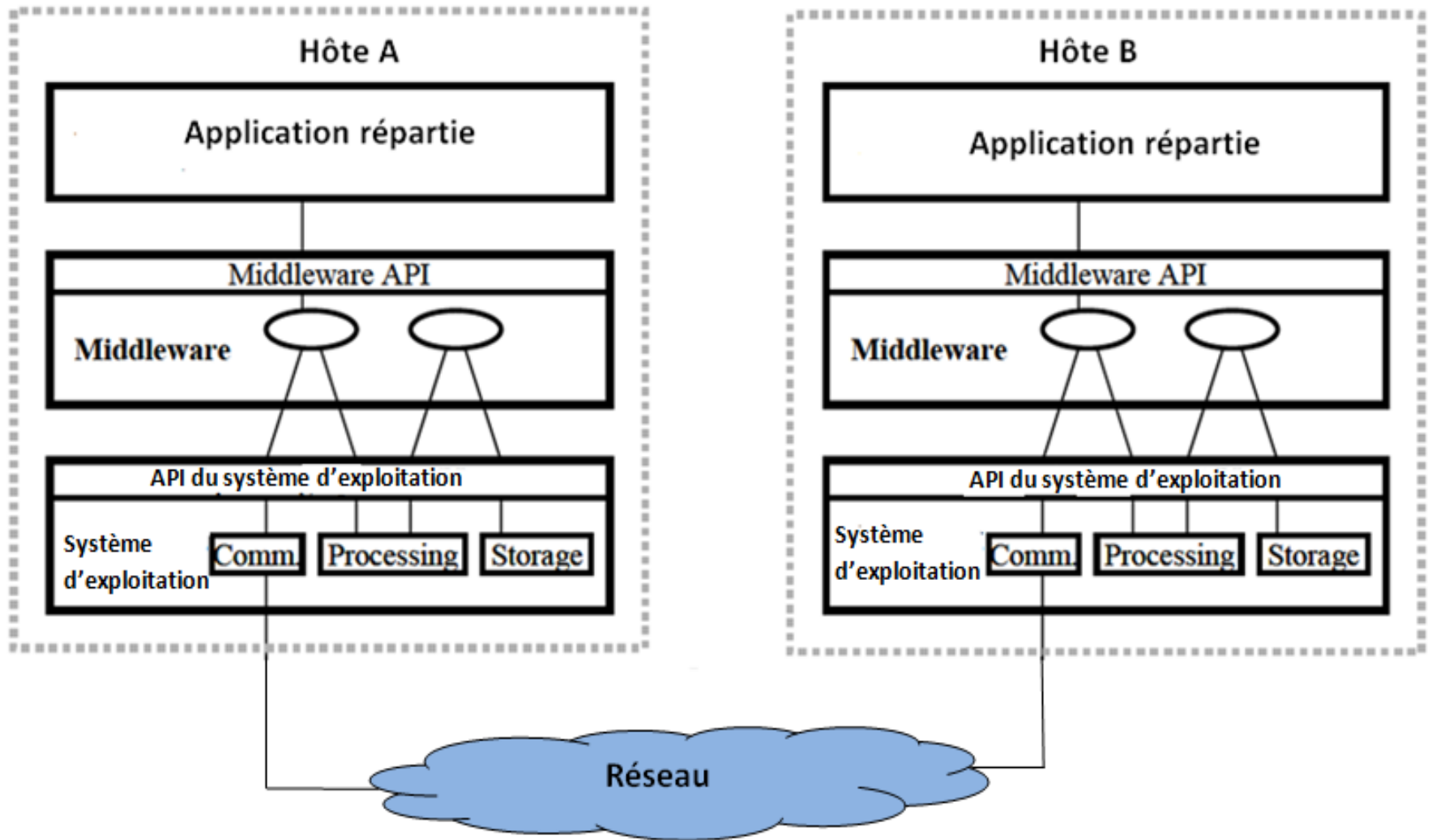




Intergiciel : est une classe de logiciels systèmes qui permet d'implanter une approche répartie:

- Fournit une API d'interaction et de communication: interactions de haut niveau pour des applications réparties.
- Fournit un ensemble de services utiles pour des applications s'exécutant en environnement réparti: désignation, cycle de vie, sécurité, transactionnel, etc.
- Fonctionne en univers ouvert (supporte des applications tournant sur des plate-formes matérielles et logicielles différentes).

Middleware (intergiciel) : Schéma Général



Application Programming Interface (API)

L'interface de programmation offerte par une couche à une couche supérieure .



■ Le middleware a quatre fonctions principales

- Fournir une interface ou API de haut niveau aux applications
- Masquer l'hétérogénéité des systèmes matériels et logiciels sous-jacents
- Rendre la répartition aussi invisible ("transparente") que possible
- Fournir des services répartis d'usage courant

■ Le middleware vise à faciliter la programmation répartie

- Développement, évolution, réutilisation des applications
- Portabilité des applications entre plates-formes
- Interopérabilité d'applications hétérogènes



- **Intergiciels à messages**, MOM: Message Oriented Middleware :
JMS (Java Message Service de J2EE de Sun), Microsoft Message
Queues Server, etc.
- **Intergiciels de bases de données** (ODBC)
- **Intergiciels à appel de procédure distante** (DCE)
- **Intergiciels à objets répartis** (CORBA, JAVA RMI, .NET Remoting)
- **Intergiciels à composants** (EJB, CCM, Web services)



1. G. Coulouris, J. Dollimore, and T. Kindberg. Distributed Systems—Concepts and Design, 2nd Ed. Addison-Wesley Publishers Ltd., 1994.
2. A. Tanenbaum. Systèmes d'exploitation : systèmes centralisés et systèmes distribués. Interéditions, Paris, 1994.
3. L. Duchien, C. Gransart, A.-F. Le Meur, P. Merle, R. Rouvoy, L. Seinturier, **Intergiciels, composants et architectures logicielles**, Université des Sciences et Technologies de Lille Master Recherche Informatique.
4. *David E. Bakken*, **MIDDLEWARE**, School of Electrical Engineering and Computer Science, *Washington State University*.
5. Sacha Krakowiak, **Middleware Adaptable**, Université Joseph Fourier Projet Sardes (INRIA et IMAG-LSR), 2004.
6. Frank Singhoff, **Introduction aux systèmes répartis**, Université de Brest, France.